

Berikut contoh lengkap soal **Multi-Prime RSA** dan **solver-nya**.

Pembuat Soal (Generator)

Misalnya kita membuat RSA dengan 3 bilangan prima.

```
from Crypto.Util.number import *
```

```
flag = b"FLAG{MULTIPRIME_RSA}"
```

```
m = bytes_to_long(flag)
```

```
p = getPrime(64)
```

```
q = getPrime(64)
```

```
r = getPrime(64)
```

```
n = p * q * r
```

```
e = 65537
```

```
c = pow(m, e, n)
```

```
print("n =", n)
```

```
print("e =", e)
```

```
print("c =", c)
```

```
# simpan untuk pembuat soal
```

```
print("p =", p)
```

```
print("q =", q)
```

```
print("r =", r)
```

Contoh output:

```
n = 3689936221228512127176635779554033941409901859346934585813
```

```
e = 65537
```

$c = 2920738533738973858347703174197268163177656897529246608537$

Yang diberikan ke peserta hanya:

$n = 3689936221228512127176635779554033941409901859346934585813$

$e = 65537$

$c = 2920738533738973858347703174197268163177656897529246608537$

Solver

Langkah 1: Faktorkan n

Untuk challenge kecil seperti ini:

```
from sympy import factorint
```

```
n = 3689936221228512127176635779554033941409901859346934585813
```

```
f = factorint(n)
```

```
print(f)
```

Misalnya output:

```
{
  16995766475411197287:1,
  16875917479342681461:1,
  12853316524139406769:1
}
```

Langkah 2: Ambil faktor-faktornya

```
primes = list(f.keys())
```

```
p = primes[0]
```

```
q = primes[1]
```

```
r = primes[2]
```

Langkah 3: Hitung $\phi(n)$

```
[
\phi(n)=(p-1)(q-1)(r-1)
]

phi = (p-1)*(q-1)*(r-1)
```

Langkah 4: Cari d

```
[
d=e^{-1}\mod\phi(n)
]

from Crypto.Util.number import inverse

d = inverse(e, phi)
```

Langkah 5: Dekripsi

```
m = pow(c, d, n)
```

Langkah 6: Konversi ke Flag

```
from Crypto.Util.number import long_to_bytes

print(long_to_bytes(m))

Output:
b'FLAG{MULTIPRIME_RSA}'
```

Solver Lengkap

```
from Crypto.Util.number import *
from sympy import factorint

n = 3689936221228512127176635779554033941409901859346934585813
e = 65537
c = 2920738533738973858347703174197268163177656897529246608537

f = factorint(n)
```

```
primes = list(f.keys())
```

```
phi = 1
```

```
for p in primes:
```

```
    phi *= (p - 1)
```

```
d = inverse(e, phi)
```

```
m = pow(c, d, n)
```

```
print(long_to_bytes(m))
```

Versi CTF yang Lebih Realistis

Biasanya pembuat challenge membuat satu faktor kecil agar peserta bisa menemukan faktornya dengan mudah:

```
from Crypto.Util.number import *
```

```
flag = b"FLAG{MULTIPRIME_RSA}"
```

```
m = bytes_to_long(flag)
```

```
p = getPrime(16)    # faktor kecil
```

```
q = getPrime(128)
```

```
r = getPrime(128)
```

```
n = p*q*r
```

```
e = 65537
```

```
c = pow(m,e,n)
```

```
print("n =",n)
```

```
print("e =",e)
```

```
print("c =",c)
```

Peserta bisa menemukan faktor kecil dengan:

```
for i in range(2,100000):
```

```
    if n % i == 0:
```

```
        print(i)
```

Setelah mendapatkan semua faktor, langkah dekripsinya sama seperti di atas. Ini adalah pola yang sering muncul pada challenge Multi-Prime RSA tingkat pemula–menengah di CTF.